CSP-S 2024 (Senior)

2024-10-26, 14:30–18:30 (4 hours)

Translated from the official Chinese statements of the second round of the 2024 CCF Certified Software Professional (CSP-J/S 2024), Senior level. The problems are given in contest order. File names in parentheses are the official problem code names. Each problem reads from `<name>.in` and writes to `<name>.out`. Merged cells of the original subtask tables are reproduced by an entry in the first row of the merged block and empty cells below it.

Problem 1: Duel (决斗)

Problem code name: `duel`. Time limit: 1 s. Memory limit: 512 MB. Total points: 100.

Problem description

Today is little Q's birthday, and he received n cards as a present. These cards belong to the popular game "Duel Monsters". The i-th card represents a monster whose attack power is r_i and whose defense power is also r_i.

A game consists of several rounds. In each round, little Q chooses a monster i and another monster j ($i \neq j$), and lets monster i attack monster j. If the attack power of monster i is less than or equal to the defense power of monster j, nothing happens. Otherwise, the defense of monster j is broken, and monster j leaves the game and no longer takes part in the rest of it. A monster can launch an attack at most once in the whole game. The game ends when every monster that has not left the game has already launched an attack.

Little Q wants to decide an order of attacks such that, when the game ends, the number of monsters that have not left the game is as small as possible.

Input

Read the input from the file `duel.in`.

The first line of the input contains a positive integer n, the number of cards.

The second line of the input contains n positive integers, the i-th of which is the attack and defense power r_i of the i-th monster.

Output

Write the output to the file `duel.out`.

Output one line containing one integer, the minimum possible number of monsters that have not left the game when the game ends.

Sample 1 input

```
5
1 2 3 1 2
```

Sample 1 output

```
2
```

Explanation of sample 1

One optimal plan is the following. In the first round, let monster 2 attack monster 1. In the second round, let monster 5 attack monster 4. In the third round, let monster 3 attack monster 5. Now every monster that has not left the game has attacked, so the game ends. It can be shown that no attack order does better.

Sample 2 input

```
10
136 136 136 2417 136 136 2417 136 136 136
```

Sample 2 output

```
8
```

Sample 3

See the files `duel/duel3.in` and `duel/duel3.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_duel3.in` and `ex_duel3.ans`.

This sample satisfies $\forall 1 \leq i \leq n,\ r_i \leq 2$.

Sample 4

See the files `duel/duel4.in` and `duel/duel4.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_duel4.in` and `ex_duel4.ans`.

Constraints

For all test data it is guaranteed that $1 \leq n \leq 10^5$ and $1 \leq r_i \leq 10^5$.

Test cases	n	r_i	Special property
$1 \sim 4$	≤ 10	$\leq 10^5$	none
$5 \sim 10$	$\leq 10^5$	≤ 2	
$11 \sim 15$	≤ 30	$\leq 10^5$	special property A
$16 \sim 20$	$\leq 10^5$		none

Special property A: it is guaranteed that every r_i is generated independently and uniformly at random from its possible range of values.

Problem 2: Speeding Detection (超速检测)

Problem code name: `detect`. Time limit: 2 s. Memory limit: 512 MB. Total points: 100.

Problem description

Little D has just joined the traffic administration of a certain country. His first task is to be in charge of speeding detection on a north–south main road of the country whose length is L. To test little D, his supervisor first asks him to solve a simplified scenario.

This weekend, n cars are expected to appear on the main road. The i-th car enters the main road at the position at distance d_i from its southern end, and drives northwards with uniform acceleration, with initial speed v_i and acceleration a_i. We only consider cars driving from south to north, so $v_i > 0$, but a_i may be positive, negative or zero. When a car reaches the northern end of the main road (that is, the position at distance L from the southern end) or its speed drops to 0 (which can only happen when $a_i < 0$), we consider that the car leaves the main road.

There are m speed cameras on the main road. The j-th speed camera is at the position at distance p_j from the southern end of the main road. Every speed camera can be set to on or off. When a car passes a speed camera that is on, if the instantaneous speed of the car exceeds the speed limit V of the road, the car is judged to be speeding. Note that when a car enters or leaves the main road, if there is a speed camera that is on at the corresponding position, that speed camera also measures the speed of the car.

The supervisor first wants to know how many of these n cars would be judged to be speeding if all speed cameras were on.

Second, to save energy, the department wants to switch off some of the speed cameras. However, they do not want to miss any speeding car. That is, if one of the n cars is judged to be speeding when all speed cameras are on, they want it to still be judged to be speeding after some speed cameras have been switched off. The supervisor also wants to know the maximum number of speed cameras that can be switched off under this condition.

Since n is very large, the supervisor allows little D to solve these two problems by programming, so little D turns to you.

If you are not familiar with acceleration, little D has kindly provided the formulas about acceleration in the "Hint" section of this problem.

Input

Read the input from the file `detect.in`.

This problem has multiple test cases.

The first line of the input contains a positive integer T, the number of test cases.

Then T test cases follow, each in the following format.

The first line contains four integers n, m, L, V: the number of cars, the number of speed cameras, the length of the main road and the speed limit of the road, respectively.

Then n lines follow. The i-th of them contains three integers d_i, v_i, a_i describing a car.

The last line contains m integers $p_1, p_2, \cdots, p_m$ describing the positions of all speed cameras on the road.

Output

Write the output to the file `detect.out`.

For each test case, output one line containing two integers. The first integer is the number of cars judged to be speeding when all speed cameras are on. The second integer is the maximum number of speed cameras that can be switched off without missing any speeding car.

Sample 1 input

```
1
5 5 15 3
0 3 0
12 4 0
1 1 4
5 5 -2
6 4 -4
2 5 8 9 15
```

Sample 1 output

```
3 3
```

Explanation of sample 1

In this test case the main road has length 15 and the speed limit is 3. There is one speed camera at each of the positions at distance $2, 5, 8, 9, 15$ from the southern end.

- The first car enters at the southern end and drives at the constant speed 3. This car is never speeding on the whole road.
- The second car enters at the position at distance 12 from the southern end and drives at the constant speed 4. When it leaves the main road at the northern end, it is judged to be speeding by the speed camera at distance 15 from the southern end.
- The third car enters at the position at distance 1 from the southern end, with initial speed 1 and acceleration 4. After it has travelled a distance of $\frac{3^2-1^2}{2\times 4} = 1$, that is, when it reaches position 2, its speed becomes 3, and it is speeding from then on. Therefore this car is judged to be speeding by every speed camera except the one at distance 2 from the southern end.
- The fourth car enters at the position at distance 5 from the southern end, with initial speed 5 and acceleration -2. After it has travelled a distance of $\frac{3^2-5^2}{2\times(-2)} = 4$, that is, when it reaches position 9, its speed becomes 3. Therefore this car is speeding while its distance from the southern end is in $[5, 9)$, and it is judged to be speeding by the two speed cameras at distance 5 and 8 from the southern end.
- The fifth car enters at the position at distance 6 from the southern end, with initial speed 4 and acceleration -4. After it has travelled a distance of $\frac{3^2-4^2}{2\times(-4)} = \frac{7}{8}$, that is, when it reaches position $6\frac{7}{8}$, its speed becomes 3. Therefore this car is speeding while its distance from the southern end is in $[6, 6\frac{7}{8})$, but there is no speed camera in this interval, so it is not judged to be speeding.

Hence the second, third and fourth cars are judged to be speeding, and the first number of the output is 3.

We can switch off the three speed cameras at distance $2, 8, 9$ from the southern end and keep the two at 5 and 15. Then the three cars that were judged to be speeding before are still judged to be speeding. It can be shown that there is no better plan, so the second number of the output is 3.

Sample 2

See the files `detect/detect2.in` and `detect/detect2.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_detect2.in` and `ex_detect2.ans`.

This sample satisfies $n, m \leq 10$.

Sample 3

See the files `detect/detect3.in` and `detect/detect3.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_detect3.in` and `ex_detect3.ans`.

This sample satisfies special property A, and its first ten test cases satisfy $n, m \leq 3000$.

Sample 4

See the files `detect/detect4.in` and `detect/detect4.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_detect4.in` and `ex_detect4.ans`.

This sample satisfies special property B, and its first ten test cases satisfy $n, m \leq 3000$.

Sample 5

See the files `detect/detect5.in` and `detect/detect5.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_detect5.in` and `ex_detect5.ans`.

This sample satisfies special property C, and its first ten test cases satisfy $n, m \leq 3000$.

Constraints

For all test data it is guaranteed that:

- $1 \leq T \leq 20$;
- $1 \leq n, m \leq 10^5$, $1 \leq L \leq 10^6$, $1 \leq V \leq 10^3$;
- $0 \leq d_i < L$, $1 \leq v_i \leq 10^3$, $|a_i| \leq 10^3$;
- $0 \leq p_1 < p_2 < \cdots < p_m \leq L$.

Test case	$n, m \leq$	Special property
1	10	none
2	20	
3	3,000	A
4	10^5	
5	3,000	B
6	10^5	
7	3,000	C
8	10^5	
9	3,000	none
10	10^5	

Special property A: it is guaranteed that $a_i = 0$.

Special property B: it is guaranteed that $a_i > 0$.

Special property C: it is guaranteed that $a_i < 0$, and that no car leaves the main road at its northern end.

Hint

The definitions and formulas related to acceleration are as follows.

- Uniformly accelerated motion is motion in which the acceleration of the object stays constant during the motion, that is, the change of speed per unit of time is constant.
- If a car has initial speed v_0 and acceleration $a \neq 0$ and moves with uniform acceleration, then after time t its speed is $v_1 = v_0 + a \times t$, and its displacement (that is, the distance travelled) is $s = v_0 \times t + 0.5 \times a \times t^2$.
- If a car has initial speed v_0 and acceleration $a \neq 0$ and moves with uniform acceleration, then when its displacement (that is, the distance travelled) is s, its instantaneous speed is $\sqrt{v_0^2 + 2 \times a \times s}$.
- If a car has initial speed v_0 and acceleration $a \neq 0$, then when its displacement (that is, the distance travelled) is $\frac{v_1^2 - v_0^2}{2a}$, its instantaneous speed is v_1.

If you compute with floating-point numbers, beware of potential precision problems.

Problem 3: Coloring (染色)

Problem code name: `color`. Time limit: 1 s. Memory limit: 512 MB. Total points: 100.

Problem description

You are given an array A of n positive integers, arranged in a row from left to right.

You need to color every number in A either red or blue, and then the final score is computed as follows.

Let C be an integer array of length n. For every number A_i in A ($1 \leq i \leq n$):

- If there is no number of the same color as A_i to its left, let $C_i = 0$.
- Otherwise, let A_j be the nearest number of the same color to the left of A_i. If $A_i = A_j$, let $C_i = A_i$; otherwise let $C_i = 0$.

Your final score is the sum of all integers in C, that is, $\sum_{i=1}^{n} C_i$. You need to maximize the final score. Find the maximum possible final score.

Input

Read the input from the file `color.in`.

This problem has multiple test cases.

The first line of the input contains a positive integer T, the number of test cases.

Then T test cases follow, each in the following format.

The first line contains a positive integer n, the length of the array.

The second line contains n positive integers $A_1, A_2, \ldots, A_n$, the elements of the array A.

Output

Write the output to the file `color.out`.

For each test case, output one line containing a non-negative integer, the maximum possible value of the final score.

Sample 1 input

```
3
3
1 2 1
4
1 2 3 4
8
3 5 2 5 1 2 1 4
```

Sample 1 output

```
1
0
8
```

Explanation of sample 1

For the first test case, here are three possible colorings.

1. Color A_1, A_2 red and A_3 blue (in the original statement the sequence 121 is printed with these colors). The score is computed as follows.

   - For A_1, there is no red number to its left, so $C_1 = 0$.
   - For A_2, the nearest red number to its left is A_1. Since $A_1 \neq A_2$, $C_2 = 0$.
   - For A_3, there is no blue number to its left, so $C_3 = 0$.

   The final score of this coloring is $C_1 + C_2 + C_3 = 0$.

2. Color all of A_1, A_2, A_3 red (the sequence 121 printed all red). The score is computed as follows.

   - For A_1, there is no red number to its left, so $C_1 = 0$.
   - For A_2, the nearest red number to its left is A_1. Since $A_1 \neq A_2$, $C_2 = 0$.
   - For A_3, the nearest red number to its left is A_2. Since $A_2 \neq A_3$, $C_3 = 0$.

   The final score of this coloring is $C_1 + C_2 + C_3 = 0$.

3. Color A_1, A_3 red and A_2 blue (the sequence 121 printed with these colors). The score is computed as follows.

   - For A_1, there is no red number to its left, so $C_1 = 0$.
   - For A_2, there is no blue number to its left, so $C_2 = 0$.
   - For A_3, the nearest red number to its left is A_1. Since $A_1 = A_3$, $C_3 = A_3 = 1$.

The final score of this coloring is $C_1 + C_2 + C_3 = 1$.

It can be shown that no coloring achieves a final score greater than 1.

For the second test case, it can be shown that every coloring has final score 0.

For the third test case, one optimal coloring is to color A_1, A_2, A_4, A_5, A_7 red and A_3, A_6, A_8 blue (in the original statement the sequence 35152124 is printed with these colors). The corresponding array is $C = [0, 0, 0, 5, 0, 1, 2, 0]$, and the final score is 8.

Sample 2

See the files `color/color2.in` and `color/color2.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_color2.in` and `ex_color2.ans`.

Constraints

For all test data it is guaranteed that $1 \le T \le 10$, $2 \le n \le 2 \times 10^5$ and $1 \le A_i \le 10^6$.

Test cases	n	A_i
$1 \sim 4$	≤ 15	≤ 15
$5 \sim 7$	$\le 10^2$	$\le 10^2$
$8 \sim 10$	$\le 2{,}000$	$\le 2{,}000$
$11, 12$	$\le 2 \times 10^4$	$\le 10^6$
$13 \sim 15$	$\le 2 \times 10^5$	≤ 10
$16 \sim 20$		$\le 10^6$

Problem 4: Arena Game (擂台游戏)

Problem code name: `arena`. Time limit: 1 s. Memory limit: 512 MB. Total points: 100.

Problem description

Little S wants to hold an arena game. If 2^k players take part in total, the game consists of k rounds:

- In the first round, the players numbered $1, 2$ play one match, the players numbered $3, 4$ play one match, and so on, up to the players numbered $2^k - 1, 2^k$ who play one match.
- In the second round, only the winners of the first round are kept, and adjacent pairs play one match each, in order.
- And so on. In round $k - 1$, only the 4 winners of round $k - 2$ are kept, and the first two and the last two play one match each, which is the so-called semifinal.
- Round k is the final between the two winners of the semifinals.

Having fixed the advancement rules, little S sets the rules of the matches to those of a "king of the hill" arena. Specifically, every player has an ability value $a_1, a_2, \cdots, a_{2^k}$, an integer in $[0, 2^{31} - 1]$. For every match, a number 0/1 is first drawn by lot. We denote the number drawn for the G-th match of round R by $d_{R,G}$. Drawing 0 means that the player with the smaller number is the defender. Drawing 1 means that the player with the larger number is the defender. The defender wins if and only if his ability value satisfies $a \ge R$. In other words, the outcome of a match depends

only on the ability value of the defender compared with the index of the current round, and not on the ability value of the other player.

Now little S has received, one after another, the registrations of n players, each of whom has told little S their ability value. Little S numbers the players $1, 2, \cdots, n$ in order of registration. Little S wants to know: after adding as few players as possible so that the total number of players is a power of 2, and after all the players play one complete arena game, what is the sum of the numbers of all players who can possibly become the overall champion?

Formally, let k be the smallest non-negative integer with $2^k \geq n$. Then $(2^k - n)$ players should be added, and the ability values of the added players can be any integers in $[0, 2^{31} - 1]$. If an added player can possibly win, he should also be counted in the answer.

Of course little S finds this problem too easy, so he gives you m queries $c_1, c_2, \cdots, c_m$. For every c_i, little S wants you to find the answer to this problem when only the registrations of the first c_i players have been received.

Input

Read the input from the file `arena.in`.

Each test file of this problem contains multiple test cases, but the different test cases are obtained only by modifying $a_1, a_2, \cdots, a_n$, while everything else stays the same. Please refer to the following format. Here $\oplus$ denotes the bitwise XOR operator, and $a \bmod b$ denotes the remainder of a divided by b.

The first line of the input contains two positive integers n, m, the number of registered players and the number of queries.

The second line of the input contains n non-negative integers $a'_1, a'_2, \cdots, a'_n$. This sequence will be used to compute the real ability values.

The third line of the input contains m positive integers $c_1, c_2, \cdots, c_m$, the queries.

Let K be the smallest non-negative integer with $2^K \geq n$. In the following K lines, the R-th line contains 2^{K-R} digits (without spaces), the G-th of which is the number $d_{R,G} = 0/1$ drawn by lot for the G-th match of round R.

Note that since a query only fills the number of players up to $2^k \geq c_i$, where $k \leq K$, you will not necessarily use all of the input values.

The next line contains a positive integer T, the number of test cases.

Then T lines follow, each describing one test case by 4 non-negative integers X_0, X_1, X_2, X_3. The ability values of this test case are $a_i = a'_i \oplus X_{i \bmod 4}$, where $1 \leq i \leq n$.

Output

Write the output to the file `arena.out`.

Output T lines in total. For each test case, let A_i be the answer to the i-th query ($1 \leq i \leq m$). You only need to output one line containing one integer, the value of $(1 \times A_1) \oplus (2 \times A_2) \oplus \cdots \oplus (m \times A_m)$.

Sample 1 input

```
5 5
0 0 0 0 0
5 4 1 2 3
1001
10
```

```
1
4
2 1 0 0
1 2 1 0
0 2 3 1
2 2 0 1
```

Sample 1 output

```
5
19
7
1
```

Explanation of sample 1

There are $T = 4$ test cases; only the first one is explained here. The real ability values of the 5 players are $[1, 0, 0, 2, 1]$. The 5 queries are about the prefixes of lengths $5, 4, 1, 2, 3$, respectively.

1. For the prefix of length 1, player 1 is the only player, so the answer is 1.
2. For the prefix of length 2, since 2 is already a power of 2, no players need to be added. By the lot $d_{1,1} = 1$, player 2 is the defender. Since $a_2 < 1$, player 1 wins, and the answer is 1.
3. For the prefix of length 3, first, player 1 wins the match between players 1 and 2 (because $d_{1,1} = 1$, so player 2 is the defender, and $a_2 < 1$). Then, although the ability value of player 4 is not yet known, player 4 surely wins the match between players 3 and 4 (because $d_{1,2} = 0$, so player 3 is the defender, and $a_3 < 1$). In the final between players 1 and 4, either may win (because $d_{2,1} = 1$, so player 4 is the defender; if $a_4 < 2$ then player 1 wins, and if $a_4 \geq 2$ then player 4 wins). Altogether, the answer is $1 + 4 = 5$.
4. For the prefix of length 4, by the analysis of the previous item, since $a_4 \geq 2$, the winner of the final is player 4.
5. For the prefix of length 5, it can be shown that the players who can possibly win are players 4, 7 and 8, so the answer is 19.

Therefore the answer for this test case is $(1 \times 19) \oplus (2 \times 4) \oplus (3 \times 1) \oplus (4 \times 1) \oplus (5 \times 5) = 5$.

Sample 2

See the files `arena/arena2.in` and `arena/arena2.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_arena2.in` and `ex_arena2.ans`.

This sample satisfies special property A.

Sample 3

See the files `arena/arena3.in` and `arena/arena3.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_arena3.in` and `ex_arena3.ans`.

This sample satisfies special property B.

Sample 4

See the files `arena/arena4.in` and `arena/arena4.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_arena4.in` and `ex_arena4.ans`.

Sample 5

See the files `arena/arena5.in` and `arena/arena5.ans` in the contestant's directory. On QOJ these are the downloadable attachment files `ex_arena5.in` and `ex_arena5.ans`.

Constraints

For all test data it is guaranteed that $2 \le n, m \le 10^5$, $0 \le a_i, X_j < 2^{31}$, $1 \le c_i \le n$ and $1 \le T \le 256$.

Test cases	$T =$	$n, m \le$	Special property A	Special property B
$1 \sim 3$	1	8	no	no
$4, 5$		500	yes	
$6 \sim 8$			no	yes
$9, 10$		5,000		no
$11, 12$		10^5	yes	
$13 \sim 15$			no	yes
$16, 17$	4			no
$18, 19$	16			
$20, 21$	64			
$22, 23$	128			
$24, 25$	256			

Special property A: it is guaranteed that every queried c_i is a power of 2.
Special property B: it is guaranteed that all $d_{R,G} = 0$.

Source

The statements were taken from the QOJ pages of the contest CSP-S 2024 (the statement PDF embedded in each page), in contest order:

- Problem 1 (duel): `https://qoj.ac/problem/10194`
- Problem 2 (detect): `https://qoj.ac/problem/10195`
- Problem 3 (color): `https://qoj.ac/problem/10196`
- Problem 4 (arena): `https://qoj.ac/problem/10197`